Northline

Line by Line: an online store in Go, PHP, MongoDB and React

A walkthrough of the go-php-mongo project: what each service does, why it is built the way it is, and the code that matters, explained line by line. Interview questions and exercises follow each part.

Part	What it covers
1. The system	Services, data ownership, events, the life of an order
2. Go platform	JWTs with the standard library, the outbox, MongoDB transactions
3. Orders and money	Pricing and tax, reserving stock, checkout, payments, refunds, sessions
4. PHP catalog	Laravel without an ORM, verifying Go's tokens, faceted search, the events worker
5. Front end and infrastructure	The API client, the payment page, nginx, Docker Compose
6. Practice	Interview questions with answers, and exercises

How to read it: every code listing shows the real file and its real line numbers. The notes under it refer to those numbers, so you can open the file side by side.

Contents

The system . . . 3

1.1 What the store does . . . 4

1.2 Four services, two languages . . . 4

1.3 Three ways services talk . . . 5

1.4 Who owns which data . . . 5

1.5 The life of an order . . . 6

Go platform code . . . 7

2.1 Verifying a JWT with the standard library . . . 8

2.2 The transactional outbox . . . 9

2.3 MongoDB transactions in driver v2 . . . 10

Orders and money . . . 11

3.1 Pricing and Canadian tax . . . 12

3.2 Never selling what you don't have . . . 13

3.3 Checkout . . . 14

3.4 When the payment arrives . . . 16

3.5 Trusting a webhook . . . 17

3.6 Refunds that can't pay out twice . . . 18

3.7 Sessions: rotation and reuse detection . . . 19

The PHP catalog . . . 21

4.1 Why no ORM . . . 22

4.2 Verifying Go's tokens in PHP . . . 22

4.3 From query string to MongoDB filter . . . 23

4.4 One aggregation for a whole listing page . . . 24

4.5 The events worker . . . 25

Front end and infrastructure . . . 26

5.1 One API client . . . 27

5.2 The payment page . . . 28

5.3 The gateway . . . 28

5.4 Starting nine containers in order . . . 29

Practice . . . 30

6.1 Interview questions . . . 31

6.2 Exercises . . . 32

Appendix: endpoints by service . . . 33

Appendix: dependencies, versions and CI . . . 34

Part 1

The system

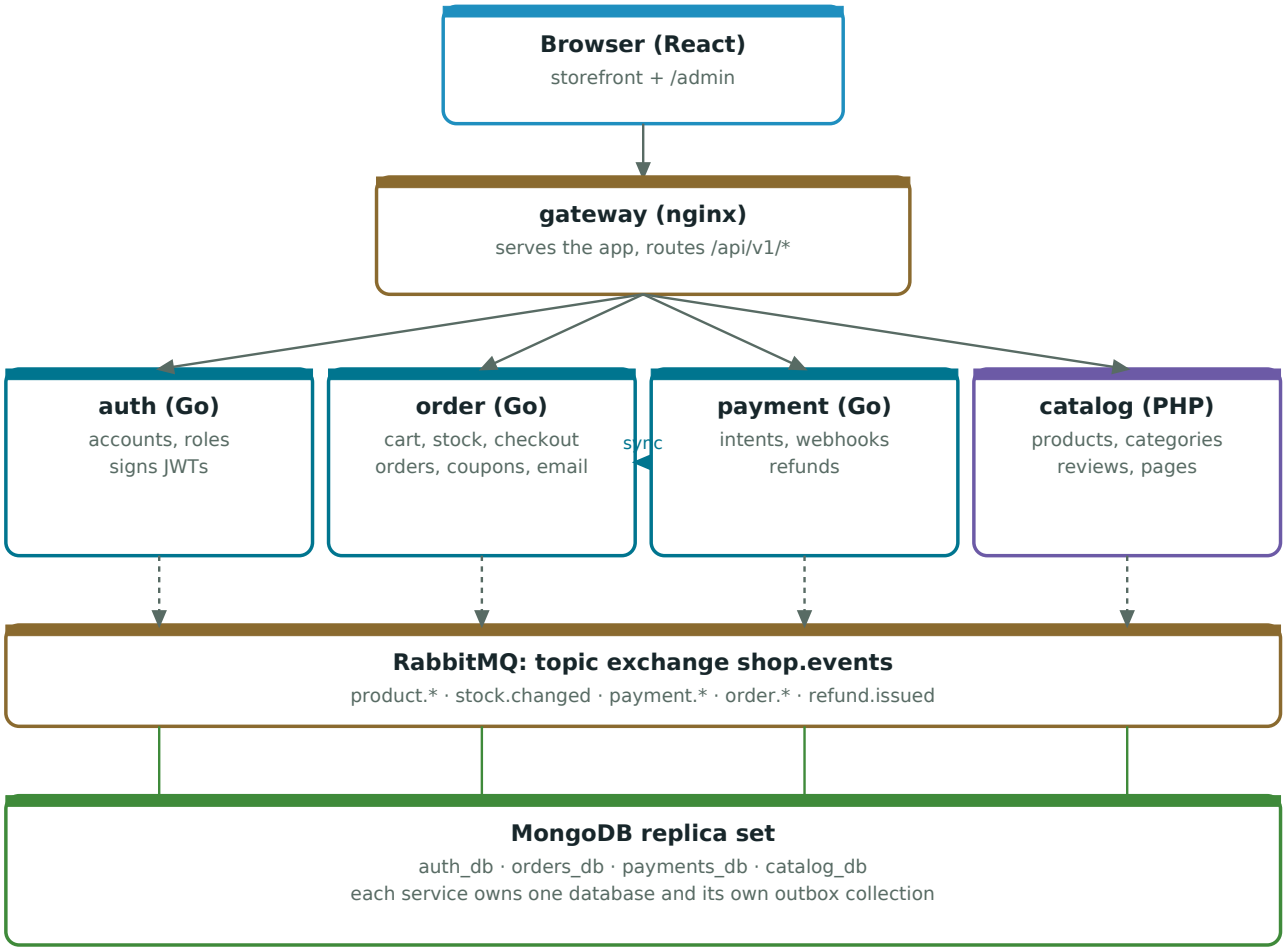
What the store does, how it's divided, and how the pieces cooperate without sharing a database.

1.1 What the store does

Northline is a complete online shop for an invented Canadian outdoor-goods brand. Customers browse a catalog, search and filter it, put items in a cart (signed in or not), check out with an address in any province, get charged the right tax, pay, receive emails, and follow their order as it ships. They can save addresses and, once they've bought something, review it.

Staff use an admin panel to create products and variants, upload images, receive stock, move orders through packing and shipping, cancel and refund, moderate reviews, edit the About/Shipping/Returns pages, make discount codes, and (admins only) grant staff access.

1.2 Four services, two languages



Service	Language	Owns	Why here
auth	Go	Users, passwords, sessions, roles. Holds the JWT signing key.	Security code with no dependencies: password hashing and RS256 are in Go's standard library.
order	Go	Carts, stock, orders, coupons, order emails.	The concurrency-sensitive core: two people buying the last unit at the same moment.

Service	Language	Owns	Why here
payment	Go	Payment attempts, provider webhooks, refunds.	Money movement: idempotency and signature checks matter most here.
catalog	PHP (Laravel 12)	Products, categories, reviews, CMS pages, uploads.	Mostly forms, validation and editorial CRUD, which Laravel makes quick.

1.3 Three ways services talk

A public key instead of a session lookup

auth-service signs short-lived access tokens with an RSA private key. The other three services only have the public key, mounted read-only from a shared Docker volume. Each one checks a token's signature itself, so a request to the catalog never waits on auth-service, and a compromised catalog can't create tokens.

Events for everything that can happen later

When something happens that another service cares about (a product changed, a payment went through), the service writes an event. RabbitMQ delivers it to every service that subscribed to that type. The sender doesn't know or care who listens, so new listeners can be added without touching it.

One synchronous call where it has to be

Before charging, payment-service must know the order's exact total right now. An event might not have arrived yet, so it asks order-service directly over HTTP at `/internal/orders/{id}`. That path is protected by a shared secret and the gateway has no route to it.

> **Rule of thumb:** use an event when the other side can act a little later and you don't need an answer; use a direct call when you need an authoritative answer before you can continue.

1.4 Who owns which data

Each service has its own MongoDB database, and no service reads another's. Where one service needs another's data, it keeps its own copy, fed by events. order-service keeps a `product_cache` with each SKU's name and price from `product.updated`; the catalog keeps each variant's `available` count from `stock.changed`.

The payoff: customers can still pay while the PHP catalog is being redeployed, and a slow catalog query can't lock up checkout. The cost: the copies are briefly out of date after a change (eventual consistency), which the design has to tolerate. Checkout, for example, always prices from order-service's own copy and never trusts a price sent by the browser.

1.5 The life of an order

Following one purchase through the system shows every moving part. Numbers are the steps in order.

#	Where	What happens
1	catalog → order	Earlier, when the product was saved, **product.updated** created stock records and the price copy in order-service.
2	browser → order	**POST /cart/items**. A guest gets a random `guest_id` cookie; the cart is keyed by it.
3	browser → order	**POST /checkout**. In one transaction: older unpaid orders by this shopper are cancelled, stock is *reserved*, the order is saved as `pending_payment`, and a **stock.changed** event goes in the outbox.
4	browser → payment → order	**POST /payments/intent**. payment-service asks order-service for the total, creates a payment with the provider, returns a client secret.
5	browser → provider → payment	The card step happens at the provider. The provider calls the **webhook**; payment-service verifies the signature and writes **payment.succeeded**.
6	payment → order	order-service consumes it: order → `paid`, reserved stock is *committed*, the cart is emptied, **order.placed** is written.
7	order → order, catalog	The email queue sends the receipt. The catalog records a verified purchase, so the buyer may review the product.
8	browser	The order page polls until it sees `paid`, then shows the confirmation.
9	admin → order	Staff move it to processing, shipped (with tracking) and delivered; each step emits an event and an email where it matters.

If the shopper never pays, a sweeper cancels the order after 15 minutes and puts the reserved units back on sale. If the money arrives after that, the late payment is refunded automatically, since the stock may already be sold to someone else.

Part 2

Go platform code

The shared building blocks every Go service uses: tokens, the outbox, MongoDB transactions.

2.1 Verifying a JWT with the standard library

A JWT is three base64url parts: header, payload, signature. The signature is RSA over SHA-256 of `header.payload`. Verifying needs only `crypto/rsa`.

Go services/go/internal/platform/token/token.go, lines 76–111

```
76  // Verify returns the claims if the signature, algorithm, issuer and expiry are all valid.
77  func (v *Verifier) Verify(tok string, now time.Time) (Claims, error) {
78      var c Claims
79      parts := strings.Split(tok, ".")
80      if len(parts) != 3 {
81          return c, ErrMalformed
82      }
83      // Pin the algorithm. Trusting the header's "alg" is the classic JWT hole:
84      // a token claiming "none" or HS256 must never reach the signature check.
85      if parts[0] != header {
86          var h struct{ Alg string }
87          raw, err := unb64(parts[0])
88          if err != nil || json.Unmarshal(raw, &h) != nil || h.Alg != "RS256" {
89              return c, ErrMalformed
90          }
91      }
92      sig, err := unb64(parts[2])
93      if err != nil {
94          return c, ErrMalformed
95      }
96      digest := sha256.Sum256([]byte(parts[0] + "." + parts[1]))
97      if err := rsa.VerifyPKCS1v15(v.pub, crypto.SHA256, digest[:], sig); err != nil {
98          return c, ErrSignature
99      }
100     payload, err := unb64(parts[1])
101     if err != nil || json.Unmarshal(payload, &c) != nil {
102         return c, ErrMalformed
103     }
104     if c.Issuer != Issuer || c.Subject == "" {
105         return c, ErrClaims
106     }
107     if now.Unix() >= c.ExpiresAt {
108         return c, ErrExpired
109     }
110     return c, nil
111 }
```

Line 79 A token must have exactly three parts; anything else is rejected before any work is done.

Lines 85–88 The most important check in the file. The classic JWT attack sends a header saying `"alg": "none"` (no signature) or HS256 (an HMAC keyed with the *public* key, which the attacker has). The algorithm is pinned to RS256 and the header is never trusted to choose it.

Line 96 The signature covers the raw base64 text exactly as received, not a re-encoding of the decoded JSON.

Line 97 RS256 means RSASSA-PKCS1-v1_5 with SHA-256. If any byte of the header or payload changed, this fails.

Line 104 The signature is valid, but was the token meant for this system? Issuer and subject are checked.

Line 107 Expiry last. `now` is a parameter, which lets tests check expiry without sleeping.

PHP does the same check with `firebase/php-jwt` (section 4.2). Because both follow the standard, a token signed in Go verifies in PHP; the project's checks confirmed this with PHP's own `openssl_verify`.

2.2 The transactional outbox

The problem: saving an order and publishing *order.placed* are two different systems (MongoDB and RabbitMQ). If the process dies between them, the order exists but nobody hears about it; publish first and die, and everyone hears about an order that doesn't exist.

The fix: write the event into an `outbox` collection in the *same database transaction* as the order. Either both are saved or neither is. A separate loop then publishes whatever is in the outbox.

Go services/go/internal/platform/events/events.go, lines 91–106

```
91  // RelayOnce publishes one batch. Exposed for tests.
92  func RelayOnce(ctx context.Context, ob Outbox, pub Publisher) error {
93      evs, err := ob.Pending(ctx, 100)
94      if err != nil {
95          return err
96      }
97      for _, ev := range evs {
98          if err := pub.Publish(ctx, ev); err != nil {
99              return err
100         }
101         if err := ob.MarkSent(ctx, ev.ID); err != nil {
102             return err // published but not marked: it goes out again, consumers dedupe
103         }
104     }
105     return nil
106 }
```

Line 93 Read up to 100 unpublished events, oldest first (the Mongo implementation sorts by `occurredAt`).

Line 98 If RabbitMQ is down, stop this batch. The same event is retried next tick, so order is preserved and nothing is skipped.

Line 101 Published but not yet marked? It will be published again. This is why delivery is *at-least-once* and why every consumer must be idempotent.

At-least-once, not exactly-once. Exactly-once delivery across a database and a broker isn't achievable in practice. Instead, every message carries an `eventId`, consumers record the IDs they've handled (the `processed_events` collection, with a 7-day TTL), and handlers are written so running twice has the same effect as running once.

2.3 MongoDB transactions in driver v2

Go services/go/internal/platform/mongox/mongox.go, lines 43–57

```
43  // Tx runs fn in a multi-document transaction. Every collection call inside fn must use
44  // the ctx it is given: in driver v2 that ctx carries the session, which is what makes the
45  // writes part of the transaction. WithTransaction also retries on transient errors
46  // (such as a write conflict with another checkout), so fn must be safe to run twice.
47  func Tx(ctx context.Context, client *mongo.Client, fn func(ctx context.Context) error) error {
48      sess, err := client.StartSession()
49      if err != nil {
50          return err
51      }
52      defer sess.EndSession(ctx)
53      _, err = sess.WithTransaction(ctx, func(ctx context.Context) (any, error) {
54          return nil, fn(ctx)
55      })
56      return err
57  }
```

Line 48 A session is the server-side context a transaction lives in.

Line 53 WithTransaction starts, commits, and retries on transient errors such as a write conflict with another checkout. That's why the callback must be safe to run more than once.

Line 54 In driver v2 the session travels inside the `ctx` passed to the callback. Any collection call made with that ctx joins the transaction; a call made with the outer ctx would silently run outside it.

Multi-document transactions require a replica set. In Docker Compose that is a one-member set: `mongod --replSet rs0`, initiated by the container's healthcheck on first start.

Part 3

Orders and money

The rules where a bug costs real money: prices, tax, stock, payments and refunds.

3.1 Pricing and Canadian tax

All money is integer cents. Floating point can't represent 0.10 exactly; summing float prices across thousands of orders drifts by cents, which accountants notice.

Go services/go/internal/order/pricing.go, lines 71–96

```go
// Quote prices a set of order lines. Order matters and matches how Canadian retailers
// charge: discount first, then shipping (free above the threshold, judged after the
// discount), then tax on the discounted goods plus shipping, since shipping is taxable.
func Quote(lines []OrderLine, coupon *Coupon, province string, now time.Time) (Totals, error) {
    var t Totals
    for _, l := range lines {
        t.SubtotalCents += l.LineTotalCents
    }
    if coupon != nil {
        d, err := coupon.Discount(t.SubtotalCents, now)
        if err != nil {
            return t, err
        }
        t.DiscountCents = d
    }
    goods := t.SubtotalCents - t.DiscountCents
    if goods < FreeShippingOverCents && len(lines) > 0 {
        t.ShippingCents = FlatShippingCents
    }
    if tax, ok := TaxRates[strings.ToUpper(province)]; ok {
        t.TaxCents = pctOf(goods+t.ShippingCents, tax.Rate)
        t.TaxLabel = tax.Label
    }
    t.TotalCents = goods + t.ShippingCents + t.TaxCents
    return t, nil
}
```

Line 76 Subtotal is the sum of line totals, which were priced from order-service's own product copy.

Lines 79–84 The coupon is checked against the subtotal (active, not expired, uses left, minimum spend). An invalid code returns a `CouponError` the checkout page shows under the field.

Line 87 Free shipping is judged *after* the discount: $80 of goods with 10% off is $72, which pays shipping. Deciding this order explicitly avoids arguments with customers.

Line 91 Tax applies to discounted goods plus shipping, since shipping is taxable in Canada.

Go services/go/internal/order/pricing.go, lines 40–43

```go
// pctOf returns amount * rate / 100000, rounded half up.
func pctOf(amount, rate int64) int64 {
    return (amount*rate + 50_000) / 100_000
}
```

Rates are stored in thousandths of a percent: Ontario's 13% is `13000` and Quebec's 14.975% is exactly 14975. Adding 50,000 before dividing by 100,000 rounds half up with integer arithmetic only, and it happens in this one function.

The order status machine

Go services/go/internal/order/pricing.go, lines 110–116

```
110 var transitions = map[string][]string{
111     StatusPendingPayment: {StatusPaid, StatusCancelled},
112     StatusPaid:           {StatusProcessing, StatusShipped, StatusCancelled, StatusRefunded},
113     StatusProcessing:     {StatusShipped, StatusCancelled, StatusRefunded},
114     StatusShipped:        {StatusDelivered, StatusRefunded},
115     StatusDelivered:      {StatusRefunded},
116 }
```

Allowed moves are data, not scattered `if` statements. `NextStatuses` filters out *paid* and *refunded*, which only payment events may set, and the admin UI shows exactly the buttons it returns.

3.2 Never selling what you don't have

Stock has two numbers: `onHand` (on the shelf) and `reserved` (promised to checkouts waiting for payment). What a new customer can buy is the difference.

Go services/go/internal/order/mongostore.go, lines 134–138

```
134 // availableAtLeast matches a stock document with at least n units free.
135 // $expr lets the filter compare two fields of the same document.
136 func availableAtLeast(n int) bson.M {
137     return bson.M{"$expr": bson.M{"$gte": bson.A{bson.M{"$subtract": bson.A{"$onHand", "$reserve
      d"}}, n}}}
138 }
```

Go services/go/internal/order/mongostore.go, lines 155–178

```
155 // Reserve is the heart of "never sell what you don't have". Each line is one
156 // conditional update: "add qty to reserved, but only if onHand - reserved >= qty".
157 // MongoDB applies a single-document update atomically, so two shoppers can't both
158 // take the last unit. It must run inside Tx: if line 3 is short, the transaction
159 // aborts and lines 1 and 2 are undone too.
160 func (s *MongoStore) Reserve(ctx context.Context, lines []Line) error {
161     var short []string
162     for _, l := range lines {
163         filter := availableAtLeast(l.Qty)
164         filter["_id"] = l.SKU
165         res, err := s.stock.UpdateOne(ctx, filter,
166             bson.M{"$inc": bson.M{"reserved": l.Qty}, "$set": bson.M{"updatedAt": time.Now().UTC
      ()}})
167         if err != nil {
168             return err
169         }
170         if res.MatchedCount == 0 {
171             short = append(short, l.SKU)
172         }
173     }
174     if len(short) > 0 {
175         return &ErrInsufficient{SKUs: short}
176     }
177     return nil
178 }
```

Line 163 The filter matches the stock document only if `onHand − reserved ≥ qty`. `$expr` lets a filter compare two fields of the same document.

Line 166 Increment `reserved` in the same operation. MongoDB applies a single-document update atomically: check and increment can't be split by another request.

Line 170 Zero matches means not enough stock. Note it and keep checking the other lines, so the error can list every short SKU.

Line 175 Returning an error aborts the surrounding transaction, which undoes the increments already made for earlier lines. All lines are reserved, or none.

> **Why not read, check, then write?** Two shoppers could both read "1 available", both pass the check, and both write. Putting the condition inside the update removes that race without locks.

Later, `Commit` moves a paid order's quantity out of both numbers (it left the shelf), `Release` gives back an expired reservation, and `Restock` returns a cancelled paid order to the shelf.

3.3 Checkout

Go services/go/internal/order/service.go, lines 264–350

```
264  // Checkout turns the cart into a pending order and reserves its stock, all in one
265  // transaction: either the order exists and its stock is held, or neither happened.
266  // Prices come from the server's own product copy; nothing the browser sends about
267  // prices is trusted. Payment happens next, in payment-service.
268  func (s *Service) Checkout(ctx context.Context, owner string, user *httpx.Principal, in Checkout
       Input) (*Order, string, error) {
269      email := strings.ToLower(strings.TrimSpace(in.Email))
270      if user != nil {
271          email = user.Email
272      }
273      fields := ValidateAddress(&in.Address)
274      if a, err := mail.ParseAddress(email); err != nil || a.Address != email {
275          fields["email"] = "enter a valid email address"
276      }
277      if len(fields) > 0 {
278          return nil, "", httpx.Invalid(fields)
279      }
280      lines, err := s.orderLines(ctx, owner)
281      if err != nil {
282          return nil, "", err
283      }
284      now := s.Now().UTC()
285      var coupon *Coupon
286      if code := normalizeCode(in.CouponCode); code != "" {
287          coupon, err = s.Store.Coupon(ctx, code)
288          if errors.Is(err, ErrNotFound) {
289              return nil, "", httpx.Invalid(map[string]string{"couponCode": "this code isn't valid
       "})
290          } else if err != nil {
291              return nil, "", err
292          }
293      }
294      totals, err := Quote(lines, coupon, in.Address.Province, now)
295      var ce *CouponError
296      if errors.As(err, &ce) {
297          return nil, "", httpx.Invalid(map[string]string{"couponCode": ce.Reason})
298      }
299      // Numbers come from a counter outside the transaction: a failed checkout leaves a gap
300      // in the sequence, which is fine, and the counter doesn't become a hot spot that
301      // every concurrent checkout transaction conflicts on.
302      n, err := s.Store.NextOrderNumber(ctx)
303      if err != nil {
304          return nil, "", err
305      }
306      o := &Order{
307          ID: id.New(), Number: fmt.Sprintf("NL-%06d", n), Owner: owner, Email: email,
```

```
308          Lines: lines, Totals: totals, Currency: "CAD", ShippingAddress: in.Address,
309          Status: StatusPendingPayment, History: []StatusChange{{Status: StatusPendingPayment, At:
             now}},
310          ReservedUntil: now.Add(ReservationTTL), CreatedAt: now, UpdatedAt: now,
311      }
312      if coupon != nil {
313          o.CouponCode = coupon.Code
314      }
315      guestKey := ""
316      if user != nil {
317          o.UserID = user.UserID
318      } else {
319          guestKey = id.Secret(24) // lets a guest view this order; only its hash is stored
320          o.GuestKeyHash = id.Hash(guestKey)
321      }
322
323      err = s.Store.Tx(ctx, func(ctx context.Context) error {
324          // A shopper who abandoned an earlier payment and checks out again shouldn't
325          // hold stock twice: release their older unpaid orders first.
326          if err := s.cancelOlderPending(ctx, owner, now); err != nil {
327              return err
328          }
329          if err := s.Store.Reserve(ctx, o.StockLines()); err != nil {
330              return err
331          }
332          if err := s.Store.InsertOrder(ctx, o); err != nil {
333              return err
334          }
335          if coupon != nil {
336              if err := s.Store.UseCoupon(ctx, coupon.Code); err != nil {
337                  return err
338              }
339          }
340          return s.emitStock(ctx, skusOf(o.StockLines()))
341      })
342      var short *ErrInsufficient
343      if errors.As(err, &short) {
344          return nil, "", httpx.Conflict("insufficient_stock", "someone just bought the last of an
             item in your cart; review your cart")
345      }
346      if err != nil {
347          return nil, "", err
348      }
349      return o, guestKey, nil
350  }
```

Line 270 A signed-in shopper's receipt goes to their account email; the form's email field is ignored for them.

Line 224 Order lines are a snapshot of the cart at current prices from order-service's product copy. Nothing price-related comes from the browser.

Line 302 Order numbers come from a counter *outside* the transaction. A failed checkout leaves a gap (harmless), and concurrent checkouts don't all conflict on one counter document.

Line 319 A guest has no account, so they get a random key that opens this order. Only its SHA-256 hash is stored, like a password.

Line 323 Everything inside commits together: cancel older unpaid orders, reserve stock, save the order, count the coupon use, queue stock.changed.

Line 326 If someone abandons a payment page and checks out again, their first order would still hold stock. It's released first.

Line 342 The store's error becomes a 409 with a message a shopper understands; the cart page then shows which lines changed.

3.4 When the payment arrives

Go services/go/internal/order/service.go, lines 384–427

```
384 // OnPaymentSucceeded marks the order paid, takes its stock off the shelf, empties the
385 // cart and announces order.placed. It is idempotent: the broker may deliver twice.
386 func (s *Service) OnPaymentSucceeded(ctx context.Context, ev events.Event) error {
387     var d paymentData
388     if !decode(ev, &d) {
389         return nil
390     }
391     now := s.Now().UTC()
392     return skipMissing(s.Store.Tx(ctx, func(ctx context.Context) error {
393         o, err := s.Store.UpdateStatus(ctx, d.OrderID, StatusUpdate{From: []string{StatusPending
      Payment},
394             To: StatusPaid, PaymentID: d.PaymentID, At: now})
395         if errors.Is(err, ErrStatus) {
396             cur, err := s.Store.Order(ctx, d.OrderID)
397             if err != nil {
398                 return err
399             }
400             if cur.Status == StatusCancelled && cur.PaymentID == "" {
401                 // The money arrived after the reservation expired and the stock was
402                 // released. Don't resurrect the order: ask payment-service to refund.
403                 slog.Warn("payment for an expired order; refunding", "order", cur.ID)
404                 return s.Store.AddEvents(ctx, events.Must(events.OrderCancelled, map[string]any{
405                     "orderId": cur.ID, "number": cur.Number, "email": cur.Email, "refund": true,
       "paymentId": d.PaymentID}))
406             }
407             return nil // already paid: a redelivery
408         }
409         if err != nil {
410             return err
411         }
412         if err := s.Store.Commit(ctx, o.StockLines()); err != nil {
413             return err
414         }
415         if err := s.Store.DeleteCart(ctx, o.Owner); err != nil {
416             return err
417         }
418         items := make([]map[string]any, len(o.Lines))
419         for i, l := range o.Lines {
420             items[i] = map[string]any{"sku": l.SKU, "productId": l.ProductID, "name": l.Name,
421                 "qty": l.Qty, "unitPriceCents": l.UnitPriceCents}
422         }
423         return s.Store.AddEvents(ctx, events.Must(events.OrderPlaced, map[string]any{
424             "orderId": o.ID, "number": o.Number, "userId": o.UserID, "email": o.Email,
425             "totalCents": o.Totals.TotalCents, "items": items}))
426     }), ev)
427 }
```

Line 388 A malformed payload can't be fixed by retrying, so it's logged and acknowledged instead of being redelivered forever.

Line 394 A compare-and-set: only an order still in `pending_payment` becomes paid. A redelivered event finds it already paid.

Line 360 The order wasn't pending. Either it's already paid (a duplicate, do nothing) or...

Line 400 ...it expired before the money arrived. Its stock may be sold, so it isn't revived. An order.cancelled event with refund=true asks payment-service to return the money.

Line 412 Reserved units leave the shelf for good.

Line 423 order.placed carries what listeners need (items, email, user), so the email sender and the catalog never have to call back.

3.5 Trusting a webhook

Anyone can POST to `/payments/webhook`. Before believing "payment succeeded", the service proves the message came from Stripe and is recent.

Go services/go/internal/payment/provider.go, lines 119–155

```
119 // VerifyStripeSignature checks the Stripe-Signature header: "t=<unix>,v1=<hex hmac>".
120 // The HMAC-SHA256 covers "<t>.<raw body>", so both the body and the time are signed,
121 // and an old captured webhook can't be replayed after the tolerance window.
122 func VerifyStripeSignature(payload []byte, header, secret string, now time.Time, tolerance time.
        Duration) error {
123     var ts int64
124     var sigs [][]byte
125     for _, part := range strings.Split(header, ",") {
126         k, v, ok := strings.Cut(strings.TrimSpace(part), "=")
127         if !ok {
128             continue
129         }
130         switch k {
131         case "t":
132             ts, _ = strconv.ParseInt(v, 10, 64)
133         case "v1":
134             if b, err := hex.DecodeString(v); err == nil {
135                 sigs = append(sigs, b)
136             }
137         }
138     }
139     if ts == 0 || len(sigs) == 0 {
140         return ErrBadSignature
141     }
142     if d := now.Sub(time.Unix(ts, 0)); d > tolerance || d < -tolerance {
143         return ErrBadSignature
144     }
145     mac := hmac.New(sha256.New, []byte(secret))
146     fmt.Fprintf(mac, "%d.", ts)
147     mac.Write(payload)
148     want := mac.Sum(nil)
149     for _, s := range sigs { // during secret rotation Stripe sends one signature per secret
150         if hmac.Equal(s, want) {
151             return nil
152         }
153     }
154     return ErrBadSignature
155 }
```

Line 131 The header looks like `t=1731100000,v1=5257a8...`. `t` is when Stripe signed it.

Line 142 A message older than five minutes is refused. Because the time is inside the signed data, an attacker can't replay an old captured webhook with a new time.

Line 145 HMAC-SHA256 with the webhook secret, over `timestamp.rawbody`. The handler reads the raw bytes; decoding and re-encoding JSON would change them.

Line 150 `hmac.Equal` compares in constant time. Several `v1` values can be present while Stripe rotates secrets.

After the signature, the event ID is recorded with a unique index (`FirstSeen`). Stripe retries webhooks, sometimes for days; the second copy is acknowledged and ignored.

3.6 Refunds that can't pay out twice

Go services/go/internal/payment/service.go, lines 191–237

```
191 // Refund returns money for an order; amount 0 means everything not yet refunded.
192 // The provider call happens first, with an idempotency key derived from the payment's
193 // refund history: if saving the result fails and staff click again, the provider
194 // returns the same refund instead of paying out twice.
195 func (s *Service) Refund(ctx context.Context, orderID string, amount int64) (*Payment, error) {
196     p, err := s.Store.ByOrder(ctx, orderID)
197     if errors.Is(err, ErrNotFound) {
198         return nil, httpx.NotFound("payment for this order")
199     }
200     if err != nil {
201         return nil, err
202     }
203     if p.Status != StatusSucceeded && p.Status != StatusPartiallyRefunded {
204         return nil, httpx.Conflict("not_refundable", "only a completed payment can be refunded")
205     }
206     remaining := p.AmountCents - p.RefundedCents
207     if amount == 0 {
208         amount = remaining
209     }
210     if amount < 0 || amount > remaining {
211         return nil, httpx.Invalid(map[string]string{"amountCents": fmt.Sprintf("refund between 0
        .01 and %d.%02d", remaining/100, remaining%100)})
212     }
213     refID, err := s.Provider.Refund(ctx, p.ProviderRef, amount, fmt.Sprintf("refund-%s-%d-%d", p
        .ID, p.RefundedCents, amount))
214     if err != nil {
215         return nil, fmt.Errorf("provider refund: %w", err)
216     }
217     full := p.RefundedCents+amount == p.AmountCents
218     to := StatusPartiallyRefunded
219     if full {
220         to = StatusRefunded
221     }
222     var out *Payment
223     err = s.Store.Tx(ctx, func(ctx context.Context) error {
224         var err error
225         out, err = s.Store.Apply(ctx, p.ID, Update{From: []string{p.Status}, To: to, At: s.Now()
        .UTC(),
226             Refund: &Refund{ID: refID, AmountCents: amount, At: s.Now().UTC()}})
227         if err != nil {
228             return err
229         }
230         return s.Store.AddEvents(ctx, events.Must(events.RefundIssued, map[string]any{
231             "paymentId": p.ID, "orderId": p.OrderID, "amountCents": amount, "full": full}))
232     })
233     if errors.Is(err, ErrStatus) {
234         return nil, httpx.Conflict("status_changed", "this payment changed meanwhile; reload")
235     }
236     return out, err
237 }
```

Line 206 Partial refunds are allowed, but never more than what's left.

Line 213 The provider call happens first, with an *idempotency key* built from the payment's refund history. If saving the result fails and staff click again, the key is the same, so Stripe returns the original refund instead of sending money twice.

Line 217 A full refund moves the payment to refunded; order-service hears refund.issued and updates the order.

3.7 Sessions: rotation and reuse detection

Access tokens last 15 minutes and live only in the browser's memory. The refresh token lasts 30 days, sits in an httpOnly cookie that JavaScript can't read, and is replaced every time it's used.

Go services/go/internal/auth/service.go, lines 107–145

```
107 // Refresh rotates a refresh token and issues a new access token with current roles,
108 // so a role change takes effect within one access-token lifetime.
109 func (s *Service) Refresh(ctx context.Context, refresh string) (*Session, error) {
110     if refresh == "" {
111         return nil, httpx.Unauthorized("no session")
112     }
113     now := s.Now()
114     t, err := s.Store.RefreshByHash(ctx, id.Hash(refresh))
115     if errors.Is(err, ErrNotFound) {
116         return nil, httpx.Unauthorized("session not found")
117     }
118     if err != nil {
119         return nil, err
120     }
121     if t.RevokedAt != nil {
122         // A retired token came back: treat the session as stolen.
123         if err := s.Store.RevokeFamily(ctx, t.FamilyID, now); err != nil {
124             return nil, err
125         }
126         return nil, httpx.Unauthorized("session revoked")
127     }
128     if now.After(t.ExpiresAt) {
129         return nil, httpx.Unauthorized("session expired")
130     }
131     u, err := s.Store.UserByID(ctx, t.UserID)
132     if err != nil {
133         return nil, httpx.Unauthorized("account not found")
134     }
135     next := id.Secret(32)
136     ok, err := s.Store.RetireRefresh(ctx, t.Hash, id.Hash(next), now)
137     if err != nil {
138         return nil, err
139     }
140     if !ok { // lost a race with another refresh using the same token
141         _ = s.Store.RevokeFamily(ctx, t.FamilyID, now)
142         return nil, httpx.Unauthorized("session revoked")
143     }
144     return s.issue(ctx, u, t.FamilyID, next)
145 }
```

Line 114 Only hashes of refresh tokens are stored. A database leak doesn't hand out live sessions.

Line 121 A token that was already exchanged is being used again. The real browser always has the newest one, so a replay means a copy was stolen.

Line 123 End the whole chain. The thief and the real user are both signed out; the real user signs in again, the thief can't.

Line 136 Retire the old token with a conditional update that only matches a live token. If two refreshes race with one token, only one wins; the other is treated as reuse.

Line 144 New access token with the user's *current* roles, so promoting someone to staff takes effect within one refresh.

Part 4

The PHP catalog

Laravel 12 for routing, validation and commands; the official MongoDB library for data.

4.1 Why no ORM

Laravel's Eloquent is built around SQL. Adapters exist for MongoDB, but they translate an ORM's ideas into document queries. This service uses the official `mongodb/mongodb` library directly, in small repository classes, so every query is the query that runs, and the product search can use an aggregation pipeline. Laravel still provides routing, validation, dependency injection, error rendering and console commands.

Business rules live in `app/Domain` as plain PHP with no framework or database imports. That's what makes them unit-testable in milliseconds: product validation, query building, category trees and the event format.

4.2 Verifying Go's tokens in PHP

PHP services/catalog/app/Http/Middleware/VerifyJwt.php, lines 40–67

```
40      /** @return array{id: string, email: string, roles: list<string>}|null */
41      private function userFrom(Request $request): ?array
42      {
43          $header = (string) $request->header('Authorization', '');
44          if ($header === '') {
45              return null;
46          }
47          if (! str_starts_with($header, 'Bearer ')) {
48              throw ApiException::unauthorized('expected a Bearer token');
49          }
50          JWT::$leeway = 5; // seconds of clock skew between containers
51          try {
52              // Passing the algorithm with the key is what pins RS256: a token whose
53              // header says "none" or "HS256" is rejected before any signature check.
54              $claims = JWT::decode(substr($header, 7), new Key(self::publicKey(), 'RS256'));
55          } catch (\Throwable $e) {
56              throw new ApiException(401, 'invalid_token', 'token: '.$e->getMessage());
57          }
58          if (($claims->iss ?? null) !== 'shop-auth' || empty($claims->sub)) {
59              throw new ApiException(401, 'invalid_token', 'token: invalid claims');
60          }
61
62          return [
63              'id' => (string) $claims->sub,
64              'email' => (string) ($claims->email ?? ''),
65              'roles' => array_values(array_map('strval', (array) ($claims->roles ?? []))),
66          ];
67      }
```

Line 44 No header means anonymous: browsing is public. A *bad* header is still a 401, so the React client knows to refresh.

Line 50 Containers' clocks can differ slightly; five seconds of slack avoids rejecting a token issued "in the future".

Line 54 The library takes the algorithm with the key, pinning RS256 exactly as the Go verifier does.

Line 58 Same issuer and subject rules as Go.

Routes choose the level: `jwt:optional` for review listings (signed-in users see a "write a review" form), `jwt` for posting a review, `jwt:staff` for the admin group and `jwt:admin` for deleting products. As in Go, admin implies staff.

4.3 From query string to MongoDB filter

PHP services/catalog/app/Domain/ProductQuery.php, lines 23–82

```php
    /**
     * @param  array<string, mixed>  $params  raw query string values
     * @param  list<string>|null  $categoryIds  the chosen category plus its descendants, or nul
      l
     * @return array{filter: array<string, mixed>, sort: array<string, int>, skip: int, limit: i
      nt, page: int, perPage: int, text: bool}
     */
    public static function build(array $params, ?array $categoryIds = null, bool $includeHidden
      = false): array
    {
        $filter = $includeHidden ? [] : ['status' => 'active'];
        if ($includeHidden && in_array($params['status'] ?? null, ProductInput::STATUSES, true))
       {
            $filter['status'] = $params['status'];
        }
        if ($categoryIds !== null) {
            // categoryIds is an array on each product; $in matches if any element matches.
            $filter['categoryIds'] = ['$in' => array_values($categoryIds)];
        }

        $min = self::dollarsToCents($params['minPrice'] ?? null);
        $max = self::dollarsToCents($params['maxPrice'] ?? null);
        // A product matches a price range if any of its variants could fall inside it.
        if ($min !== null) {
            $filter['priceMax'] = ['$gte' => $min];
        }
        if ($max !== null) {
            $filter['priceMin'] = ['$lte' => $max];
        }
        if (($params['inStock'] ?? '') === '1') {
            $filter['inStock'] = true;
        }

        $q = trim((string) ($params['q'] ?? ''));
        $text = false;
        if ($q !== '') {
            // Uses the text index on name, description and brand. Text search
            // understands word stems ("hoodies" finds "hoodie") and ranks by relevance.
            $filter['$text'] = ['$search' => mb_substr($q, 0, 100)];
            $text = true;
        }

        $sortKey = (string) ($params['sort'] ?? '');
        if ($text && ($sortKey === '' || $sortKey === 'relevance')) {
            $sort = ['score' => ['$meta' => 'textScore']];
        } else {
            $sort = self::SORTS[$sortKey] ?? self::SORTS['featured'];
        }
        $sort['_id'] = 1; // a unique tiebreaker, so pages never repeat or skip items

        $perPage = (int) ($params['perPage'] ?? 12);
        $perPage = max(1, min($perPage, self::MAX_PER_PAGE));
        $page = max(1, (int) ($params['page'] ?? 1));

        return [
            'filter' => $filter,
            'sort' => $sort,
            'skip' => ($page - 1) * $perPage,
            'limit' => $perPage,
            'page' => $page,
            'perPage' => $perPage,
            'text' => $text,
        ];
    }
```

Line 30 The storefront only ever sees active products; the admin list can see drafts and archived ones.

Line 36 The controller passes the chosen category plus its descendants, so "Apparel" includes T-shirts and Hoodies.

Line 43 Each product stores its cheapest and dearest variant. A product belongs in a $20–$50 range if its range overlaps it: priceMax ≥ 20 and priceMin ≤ 50.

Line 57 Text search uses the weighted index (name 10, brand 5, description 1) and understands stems: "hoodies" finds "hoodie".

Line 67 A unique tiebreaker. Without it, products with equal sort keys can swap places between page loads, so page 2 repeats or skips items.

4.4 One aggregation for a whole listing page

PHP services/catalog/app/Data/ProductRepository.php, lines 17–47

```
17      /**
18       * Runs a ProductQuery in one aggregation: $facet returns the page of items, the
19       * total for pagination and the price range for the filter slider together.
20       *
21       * @param  array<string, mixed>  $q  from ProductQuery::build
22       * @return array{items: list<array<string, mixed>>, total: int, priceMin: int, priceMax: int
          }
23       */
24      public function search(array $q): array
25      {
26          $pipeline = [
27              ['$match' => $q['filter']],
28              ['$facet' => [
29                  'items' => [
30                      ['$sort' => $q['sort']],
31                      ['$skip' => $q['skip']],
32                      ['$limit' => $q['limit']],
33                      ['$project' => ['description' => 0]], // listings don't need the long text
34                  ],
35                  'total' => [['$count' => 'n']],
36                  'price' => [['$group' => ['_id' => null, 'min' => ['$min' => '$priceMin'], 'max'
     => ['$max' => '$priceMax']]]],
37              ]],
38          ];
39          $res = iterator_to_array($this->products->aggregate($pipeline), false)[0] ?? [];
40
41          return [
42              'items' => $res['items'] ?? [],
43              'total' => $res['total'][0]['n'] ?? 0,
44              'priceMin' => $res['price'][0]['min'] ?? 0,
45              'priceMax' => $res['price'][0]['max'] ?? 0,
46          ];
47      }
```

Line 27 Filter first, so every later stage works on matching products only (and the indexes are used).

Line 29 `$facet` runs several sub-pipelines over the same matched documents. This one sorts, skips, limits and drops the long description.

Line 35 The total for "Page 2 of 5", from the same match, in the same round trip.

Line 36 The price range of everything matching, for the filter's placeholder values.

4.5 The events worker

PHP serves each web request and forgets it, so long-running work belongs in a separate process. `php artisan events:work` runs in its own container from the same image.

PHP services/catalog/app/Console/Commands/EventsWork.php, lines 57–76

```
57          $channel->basic_consume(self::QUEUE, '', false, false, false, false, function (AMQPMessa
        ge $msg) use ($handlers, $outbox) {
58              $event = json_decode($msg->getBody(), true);
59              if (! is_array($event) || ! isset($event['eventId'], $event['type'])) {
60                  $msg->nack(false); // undecodable: retrying can't help
61                  return;
62              }
63              if ($outbox->seen($event['eventId'])) {
64                  $msg->ack();
65                  return;
66              }
67              try {
68                  ($handlers[$event['type']] ?? fn () => null)($event['data'] ?? []);
69                  $outbox->markProcessed($event['eventId']);
70                  $msg->ack();
71              } catch (\Throwable $e) {
72                  $this->error("handler failed for {$event['type']}: {$e->getMessage()}");
73                  sleep(1);
74                  $msg->nack(true); // requeue
75              }
76          });
```

Line 60 Garbage can't be fixed by retrying: reject without requeueing.

Line 63 Already handled: acknowledge and move on. This is the PHP side of at-least-once delivery.

Line 74 A real failure (MongoDB unreachable, say) puts the message back for another try.

PHP services/catalog/app/Console/Commands/EventsWork.php, lines 79–91

```
79          while (true) {
80              $this->relay($outbox, $channel);
81              try {
82                  $channel->wait(null, false, 1.0);
83              } catch (AMQPTimeoutException) {
84                  // no message this second; loop round to the outbox again
85              }
86              if (! $connection->isConnected()) {
87                  $this->error('RabbitMQ connection lost; exiting so Docker restarts the worker');
88
89                  return self::FAILURE;
90              }
91          }
```

One loop alternates between publishing the catalog's outbox and waiting up to one second for incoming messages. If the connection drops, the command exits and Docker's restart policy starts a fresh worker, which is simpler and safer than reconnect logic.

Part 5

Front end and infrastructure

How the React app talks to four services as one, and how nine containers start in the right order.

5.1 One API client

TypeScript web/src/api/client.ts, lines 60–76

```
60  /** Exchanges the refresh cookie for a new access token. Concurrent callers share one request. *
        /
61  export function refreshSession(): Promise<Session | null> {
62    refreshing ??= (async () => {
63      try {
64        const res = await send("POST", "/auth/refresh");
65        const s = res.ok ? ((await res.json()) as Session) : null;
66        setSession(s);
67        return s;
68      } catch {
69        setSession(null);
70        return null;
71      } finally {
72        refreshing = null;
73      }
74    })();
75    return refreshing;
76  }
```

Line 62 If five requests get a 401 at once, they share one refresh request instead of starting five (which would also trip reuse detection).

Line 64 No token is sent; the browser attaches the httpOnly refresh cookie by itself, because its path is /api/v1/auth.

Line 66 Listeners (the auth context) update the signed-in user everywhere.

TypeScript web/src/api/client.ts, lines 78–86

```
78  export async function api<T>(method: string, path: string, body?: unknown): Promise<T> {
79    let res = await send(method, path, body);
80    if (res.status === 401 && accessToken && !path.startsWith("/auth/")) {
81      if (await refreshSession()) res = await send(method, path, body);
82    }
83    if (!res.ok) throw await toError(res);
84    if (res.status === 204) return undefined as T;
85    return (await res.json()) as T;
86  }
```

Line 80 Only retry when there was a token to expire, and never for the auth endpoints themselves.

Line 83 Every service returns errors in the same shape, so one function turns them into an ApiError with per-field messages for the forms.

5.2 The payment page

Pay in `pages/shop/Checkout.tsx` asks payment-service for an intent. With the built-in test provider it shows "Pay" and "Simulate a declined card"; with Stripe it lazy-loads Stripe's script and mounts its PaymentElement, an iframe where card numbers are typed straight into Stripe. Either way the app then navigates to the order page, which polls every 1.5 seconds while the order is still `pending_payment`, because the confirmation travels payment-service → RabbitMQ → order-service.

5.3 The gateway

nginx gateway/nginx.conf, lines 15–19

```
15      resolver 127.0.0.11 valid=10s ipv6=off;
16      set $auth    http://auth:8080;
17      set $order   http://order:8080;
18      set $payment http://payment:8080;
19      set $catalog http://catalog:8000;
```

Line 15 Docker's internal DNS. With service addresses in variables, nginx resolves them per request (cached 10 s): it starts even if a service is still building, and follows a restarted container to its new IP.

nginx gateway/nginx.conf, lines 32–52

```
32      # Slow down password guessing: 10 attempts a minute per IP, with a small burst.
33      location ~ ^/api/v1/auth/(login|register|forgot-password|reset-password)$ {
34          limit_req zone=signin burst=5 nodelay;
35          proxy_pass $auth;
36      }
37
38      # ---------- auth-service (Go) ----------
39      location ~ ^/api/v1/(auth|me)(/|$)            { limit_req zone=api burst=60 nodelay; proxy_p
          ass $auth; }
40      location ~ ^/api/v1/admin/users(/|$)          { proxy_pass $auth; }
41
42      # ---------- order-service (Go) ----------
43      location ~ ^/api/v1/(cart|checkout|orders)(/|$) { limit_req zone=api burst=60 nodelay; proxy
          _pass $order; }
44      location ~ ^/api/v1/admin/(orders|inventory|coupons|stats)(/|$) { proxy_pass $order; }
45
46      # ---------- payment-service (Go) ----------
47      location ~ ^/api/v1/payments(/|$)             { limit_req zone=api burst=60 nodelay; proxy_p
          ass $payment; }
48      location ~ ^/api/v1/admin/payments(/|$)       { proxy_pass $payment; }
49
50      # ---------- catalog-service (PHP) ----------
51      location ~ ^/api/v1/(catalog|content)(/|$)    { limit_req zone=api burst=100 nodelay; proxy_
          pass $catalog; }
52      location ~ ^/api/v1/admin/(catalog|content)(/|$) { proxy_pass $catalog; }
```

Sign-in endpoints get a stricter rate limit (10 a minute per IP) than the rest. Routing is by path prefix only; the gateway knows nothing about tokens, which each service checks itself.

5.4 Starting nine containers in order

Container	Waits for	Why
mongo	—	Its healthcheck also initiates the replica set on first start.
rabbitmq, mailpit	—	
auth	mongo healthy	Creates the JWT key pair in the shared volume, then reports healthy.
order, payment	auth + rabbitmq healthy	They need the public key and the broker.
catalog	auth + mongo healthy	Creates indexes, seeds the demo store once, then serves.
catalog-worker	catalog healthy	Indexes exist; seeded events are waiting in the outbox.
gateway	the four services started	Builds and serves the React app; routes the API.

Both the Go and the PHP Dockerfiles run their test suites while building. An image is only produced when the tests pass, so `docker compose up --build` is also a test run.

Part 6

Practice

Questions an interviewer might ask about this project, with the answers the code supports, then exercises.

6.1 Interview questions

Why split the backend into services at all? What does it cost?

Each service can be deployed, scaled and changed on its own, and a failure in one (the catalog) doesn't stop another (checkout). The costs are real: data is duplicated and briefly inconsistent, a request may cross several processes, and there's more infrastructure (a broker, more containers). For a small team a modular monolith is often the better start; this project shows the patterns that become necessary once you split.

How do you stop two customers buying the last item?

The stock update is conditional: increment *reserved* only if $onHand - reserved \geq qty$, in a single atomic document update. For multi-line carts, all lines are reserved inside one transaction, so a shortage on any line rolls back the others.

What is the transactional outbox and what problem does it solve?

Writing data and publishing a message are two systems, so one can succeed without the other. The outbox writes the message into the same database transaction as the data, and a relay publishes it afterwards. It guarantees the message goes out at least once if, and only if, the data was saved.

If messages can arrive twice, how is double-processing prevented?

Every event has an ID that consumers record after handling. Handlers are also idempotent by construction: the paid transition is a compare-and-set from pending_payment, product upserts write the same values again, and stock records are created with $setOnInsert so a replay never resets counted stock.

Why RS256 rather than HS256 for the JWTs?

With HS256 every service that verifies tokens holds the same secret and could therefore create tokens. With RS256 only auth-service holds the private key; the others hold a public key that can only verify.

Where is the access token stored in the browser, and why?

In a JavaScript variable, never localStorage, so an injected script can't read it from storage. The long-lived refresh token is an httpOnly, SameSite=Strict cookie scoped to /api/v1/auth: script can't read it and other sites can't make the browser send it.

How do you know a webhook really came from Stripe?

Stripe signs a timestamp plus the raw body with HMAC-SHA256 using a shared secret. The service recomputes it, compares in constant time, rejects messages older than five minutes, and ignores event IDs it has seen.

Why store money as integers?

Binary floating point can't represent most decimal fractions exactly, so sums drift. Cents as integers are exact; tax rates in thousandths of a percent keep even Quebec's 14.975% exact, with rounding done once, explicitly.

What happens if the catalog service is down?

Browsing and search fail, but carts, checkout and payment keep working: order-service prices from its own product copy. Events the catalog misses wait in its RabbitMQ queue and are processed when it comes back.

Why does payment-service call order-service directly instead of using an event?

It needs the authoritative amount *before* it can create a payment. An event might not have arrived yet. Direct calls are the right tool when you need an answer to continue; events are for things that can happen later.

6.2 Exercises

- **Wishlist.** Add a wishlist to auth-service (or a new service). Decide which service should own it and what events, if any, it needs. Add a heart button on product pages.
- **Low-stock alerts.** When a SKU's available stock crosses its low-stock threshold, email staff. Which service detects it, and how do you avoid sending the same alert on every sale below the threshold?
- **Search typo tolerance.** MongoDB text search doesn't handle typos. Add Meilisearch or MongoDB Atlas Search, fed by product.updated events, and compare results for "hoody".
- **Dead-letter queue.** A handler that always fails is requeued forever. Configure a dead-letter exchange and a retry limit in RabbitMQ, and add an admin page listing dead letters.
- **Kubernetes.** Write manifests like the other two Full Stack projects: Deployments, Services, a StatefulSet for MongoDB, Secrets for the keys, and an Ingress replacing the nginx gateway.
- **Contract tests.** Write a JSON Schema for each event in a shared folder, and test that the Go and PHP producers emit events that match it.
- **Tax configuration.** Move tax rates out of code into an admin-editable collection with effective dates, so a rate change can be scheduled ahead of time.

Appendix: endpoints by service

Path (under /api/v1)	Service	Who
auth/register · login · refresh · logout · forgot-password · reset-password	auth	anyone
me · me/password · me/addresses	auth	signed in
admin/users	auth	admin
cart · cart/items · cart/merge · checkout/quote · checkout	order	anyone (merge: signed in)
orders · orders/{id}	order	owner, guest key, or staff
admin/orders · admin/inventory · admin/stats · admin/coupons	order	staff (saving coupons: admin)
payments/config · payments/intent · payments/{id} · payments/mock/confirm · payments/webhook	payment	anyone (intent checks ownership)
admin/payments · admin/payments/refund	payment	staff (refund: admin)
catalog/products · catalog/products/{slug} · catalog/categories · content/pages/{slug}	catalog	anyone
catalog/products/{id}/reviews	catalog	read: anyone; write: verified buyers
admin/catalog/* · admin/content/*	catalog	staff (delete product: admin)

Appendix: dependencies, versions and CI

Three languages means three package managers, and each has its own lock file. The idea is the same in all of them: the manifest says what's allowed, the lock records exactly what was installed, and CI installs from the lock.

	Manifest (allowed)	Lock (exact, with hashes)	Install exactly
Go services	go.mod	go.sum	go mod download, then go mod verify
PHP catalog	composer.json	composer.lock	composer install
React storefront	package.json	package-lock.json	npm ci

Go: minimum version selection

go.mod lists each module at a minimum version. Go doesn't take the newest release allowed; it takes the *highest minimum* any module in the build asks for, so a build is reproducible even without a lock. go.sum adds a checksum for every module, and `go mod verify` fails if a downloaded module doesn't match. A major version is part of the import path (`go.mongodb.org/mongo-driver/v2`), so a breaking upgrade is a deliberate code change, never a silent update.

PHP: Composer

composer.json uses caret ranges like npm (`"laravel/framework": "^12.0"`: any 12.x). With composer.lock committed, `composer install` ignores the ranges and installs the locked versions; `composer update` is the deliberate step that moves them. `config.platform` tells Composer to resolve as if PHP were 8.3.0, the oldest supported, so a developer on a newer PHP can't lock a package that needs it.

The workflow

YAML ../../.github/workflows/fullstack-go-php-mongo.yml, lines 18–33

```yaml
  go:
    name: go services
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Full Stack/go-php-mongo/services/go
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-go@v5
        with:
          go-version-file: Full Stack/go-php-mongo/services/go/go.mod
          cache-dependency-path: Full Stack/go-php-mongo/services/go/go.sum
      # go.sum pins every module by checksum; verify fails if anything downloaded doesn't match.
      - run: go mod verify
      - run: go vet ./...
      - run: go test -race ./...
```

Line 28 The Go version comes from go.mod's go line, so CI and developers agree.

Line 32 Static checks the compiler doesn't make: wrong printf verbs, copied locks, unreachable code.

Line 33 The race detector instruments memory access and fails a test that reads and writes the same data from two goroutines without synchronization. The relays and consumers are concurrent, so this matters here.

YAML ../../.github/workflows/fullstack-go-php-mongo.yml, lines 35–52

```yaml
  catalog:
    name: php catalog
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Full Stack/go-php-mongo/services/catalog
    steps:
      - uses: actions/checkout@v7
      - uses: shivammathur/setup-php@v2
        with:
          php-version: "8.3"
          extensions: mongodb, sockets
          tools: composer:v2
      # With a lock file, composer install installs exactly the locked versions.
      - run: composer validate --no-check-publish
      - run: composer install --no-scripts --no-interaction --prefer-dist
      - run: vendor/bin/phpunit
      - run: composer audit --locked --no-dev
```

Line 46 PHP extensions the catalog needs: the MongoDB driver and sockets for RabbitMQ.

Line 49 Fails if composer.json is malformed or the lock is out of date with it.

Line 52 Known vulnerabilities in the locked runtime packages fail the job.

YAML ../../.github/workflows/fullstack-go-php-mongo.yml, lines 71–81

```yaml
  images:
    name: docker images
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Full Stack/go-php-mongo
    steps:
      - uses: actions/checkout@v7
      # compose reads .env for some settings; the example values are fine for a build.
      - run: cp .env.example .env
      - run: docker compose build
```

The storefront job runs `npm ci`, the build and `npm audit`, and the images job builds every Dockerfile the way Compose does. Like the other Full Stack workflows, it runs on every push to every branch and every pull request. Dependabot opens weekly pull requests for go.mod, composer.json and package.json, and monthly ones for the Dockerfiles' base images; this workflow tests each one before it's merged.

Why does Go not need ranges like ^1.10?

Minimum version selection: each module states the oldest version it works with, and the build uses the highest of those. Nothing changes until someone raises a minimum, so builds don't drift.

composer install vs composer update?

install follows the lock; update re-resolves the ranges and rewrites it. CI and Docker only ever install.